

BLUEJAY : A WEB & AI POINT OF SALE SYSTEM

Team 1

Iteration 1



TEAM MEMBERS

SARA ORION

Team Leader & Security
Leader



**KRIZMA
NAGI**

Requirement Leader



**KIMLENG
LIM**

Design & Implementation
Leader



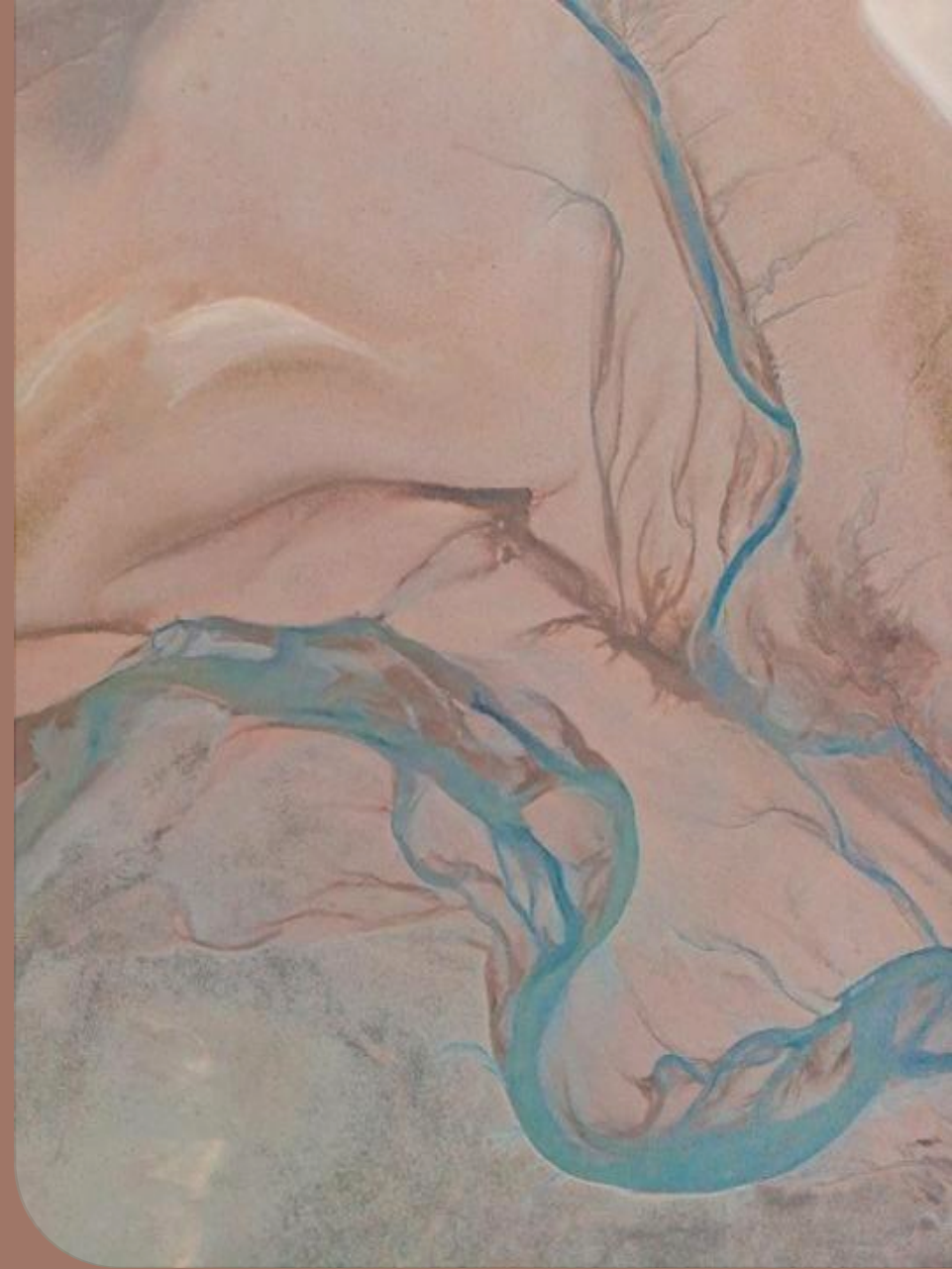
**ANDY
CHANG**

Configuration Leader



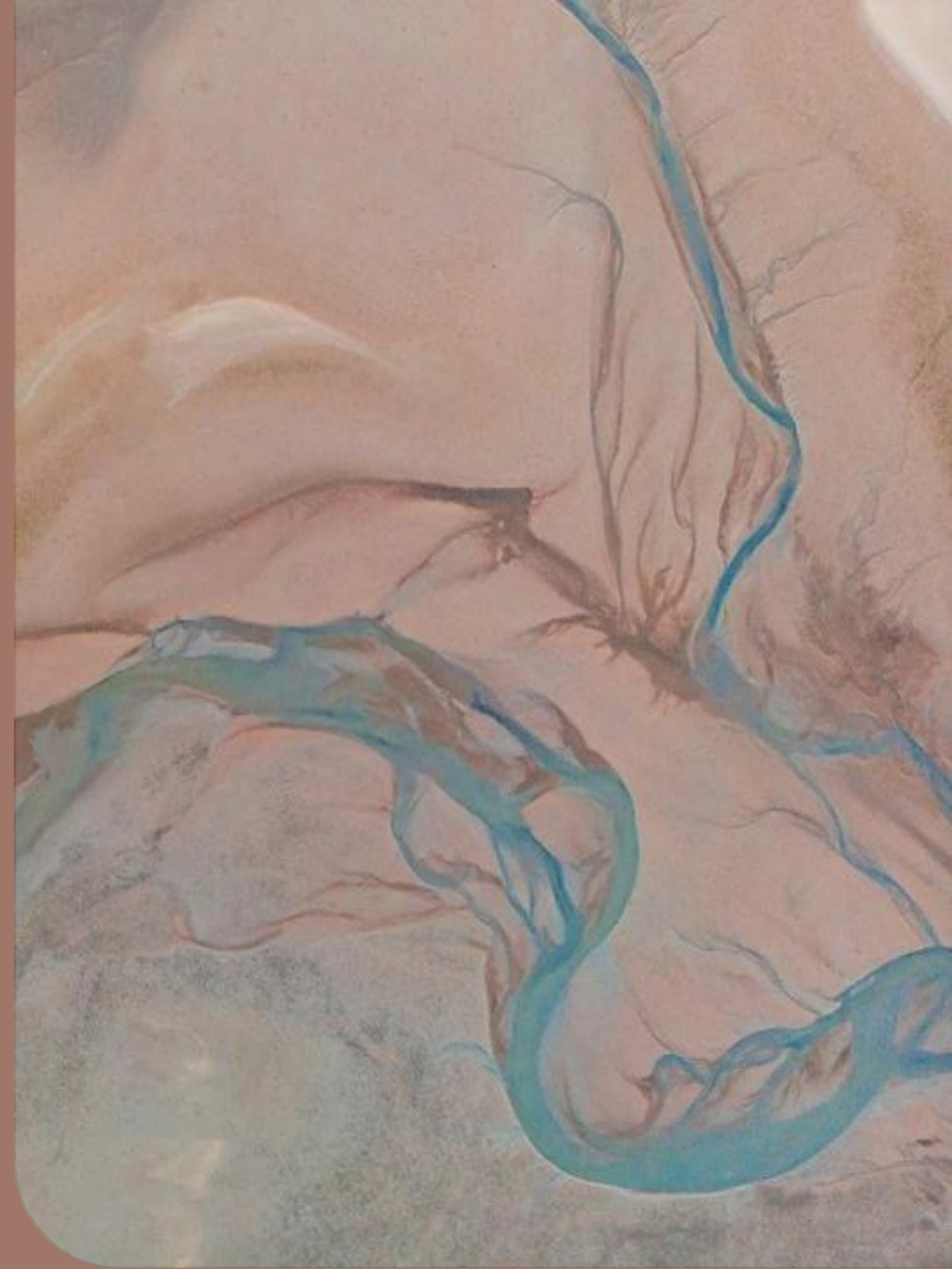
ITALIA TRAN

QA Leader



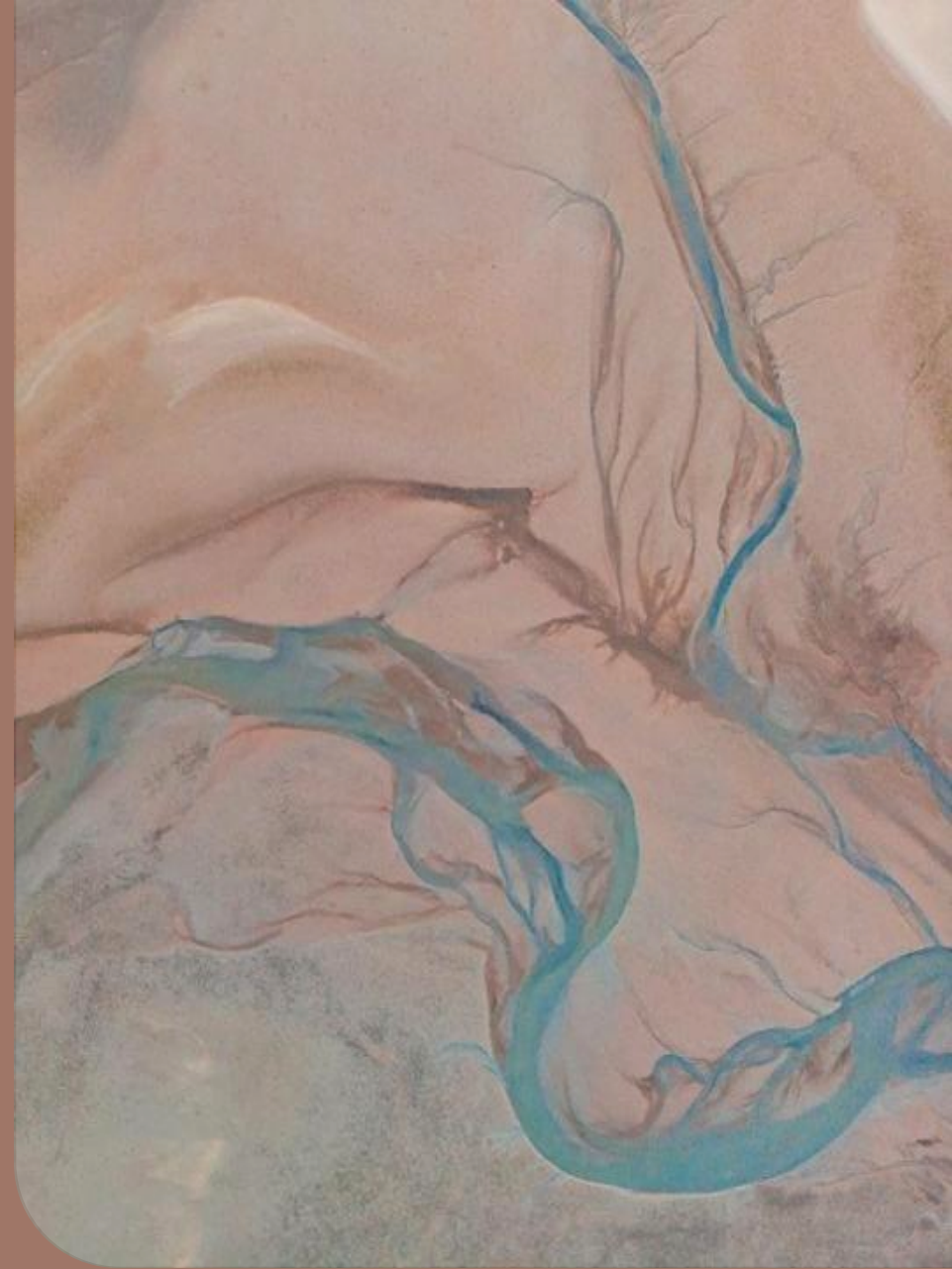
ITERATION 0 RECAP

- Finalized team roles and responsibilities for each person
- Get everyone set up on Github
- Decided to enhance Kimleng's current console based POS system into a full-fledged web application
- Designed the requirements for the new application and assigned who is doing what



ITERATION 1 OVERVIEW

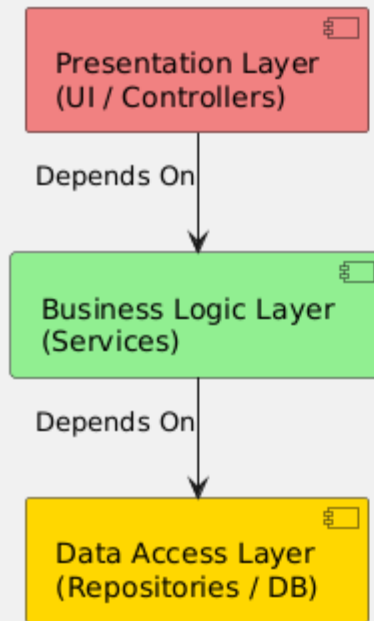
- Chose MySQL and set up a three-container Docker environment: MySQL 8.4, Spring Boot (Java 21), and Angular.
- Documented a Clean Architecture approach and established a Docker Compose workflow for development
- Successfully deployed authentication screen: login, logout, and main screen
- Backend: Spring Boot scaffold, JWT authentication with Spring Security, CORS, API docs via Scalar/PlantUML, and JUnit tests in CI
- Frontend: Angular scaffold, CORS proxy setup, login form, Docker integration
- DevOps: GitHub Actions CI/CD for the backend
- Confirmed testing policy for both unit and system testing and structured format on how the system testing will be created and ran



ARCHITECTURE DESIGN

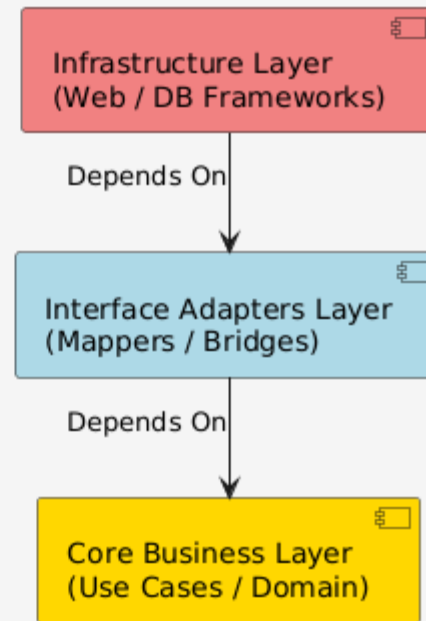
Architectural Evolution: Layered vs. Clean Architecture (Bluejay)

Traditional Layered Architecture



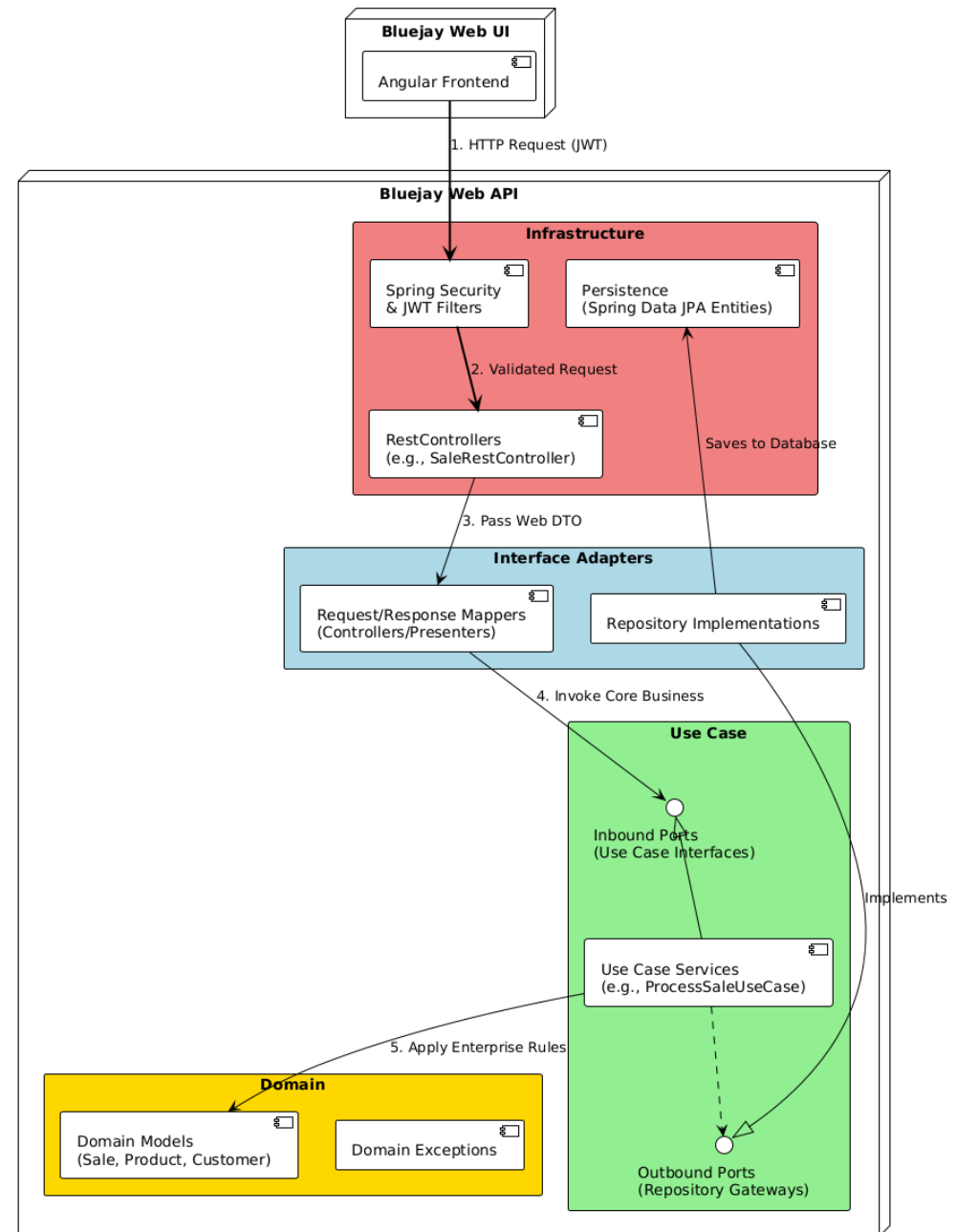
Core business depends on DB

Project Bluejay (Clean Architecture)



Frameworks depend on Core Business

Bluejay High-Level Architecture



```
classDiagram
    class SecurityConfig {
        <<config>>
        jwtAuthenticationFilter: JwtAuthenticationFilter
        securityFilterChain(HttpSecurity): SecurityFilterChain
        corsConfigurationSource(): CorsConfigurationSource
        passwordEncoder(): PasswordEncoder
        authenticationManager(AuthenticationConfiguration): AuthenticationManager
    }
    class JwtAuthenticationFilter {
        <<filter>>
        tokenProvider: JwtTokenProvider
        userDetailsService: UserDetailsService
        AUTHORIZATION_HEADER: String
        BEARER_PREFIX: String
        doFilterInternal(HttpServletRequest, HttpServletResponse, FilterChain): void
        parseJwt(HttpServletRequest): String?
    }
    class AuthController {
        <<controller>>
        authService: AuthService
        login(LoginRequest): ResponseEntity<ApiResponse<AuthResponse>>
    }
    class AuthService {
        <<interface>>
        login(LoginRequest): AuthResponse
    }
    class CustomUserDetails {
        <<record>>
        +userId: UUID
        +username: String
        +passwordHash: String
        +enabled: boolean
        +authorities: Set<GrantedAuthority>
        extractAuthorities(User): Set<GrantedAuthority>
    }
    class JwtTokenProvider {
        <<provider>>
        key: SecretKey
        expirationMs: long
        generateToken(Authentication): String
        getUsernameFromToken(String): String
        validateToken(String): boolean
    }
    class UserDetailsServiceImpl {
        <<impl>>
        userRepository: UserRepository
        loadUserByUsername(String): UserDetails
    }
    class UserRepository {
        <<interface>>
        findByUsernameWithRolesAndPermissions(String): Optional<User>
    }
    class AuthResponse {
        <<record>>
        +username: String
        +token: String
        +tokenType: String
        +expiresIn: Long
        of(String, String, Long): AuthResponse
    }
    class LoginRequest {
        <<record>>
        +username: String
        +password: String
    }
    class User {
        <<table>>
        +id: UUID <<PK>>
        +username: String <<UQ, Not Null>>
        +passwordHash: String <<Not Null>>
        +enabled: Boolean <<Not Null>>
        +createdAt: LocalDateTime <<Not Null>>
        +roles: Set<Role>
        builder(): UserBuilder
    }
    class Role {
        <<table>>
        +id: Long <<PK, AutoIncrement>>
        +name: String <<UQ, Not Null>>
        +description: String
        +permissions: Set<Permission>
        builder(): RoleBuilder
    }
    class Permission {
        <<table>>
        +id: Long <<PK, AutoIncrement>>
        +name: String <<UQ, Not Null>>
        +description: String
        builder(): PermissionBuilder
    }

    SecurityConfig --> JwtAuthenticationFilter : jwtAuthenticationFilter
    JwtAuthenticationFilter --> JwtTokenProvider : tokenProvider
    JwtAuthenticationFilter --> UserDetailsServiceImpl : userDetailsService
    AuthController --> AuthService : authService
    AuthService --> AuthResponse : returns
    CustomUserDetails --> JwtTokenProvider : tokenProvider
    CustomUserDetails --> UserRepository : queries
    CustomUserDetails --> User : constructs from
    JwtTokenProvider --> User : constructs from
    UserDetailsServiceImpl --> UserRepository : userRepository
    UserRepository --> User : retrieves
    AuthResponse --> LoginRequest : accepts
    LoginRequest --> AuthResponse : returns
    User --> Role : user_roles (user_id <FK>, role_id <FK>)
    Role --> Permission : role_permissions (role_id <FK>, permission_id <FK>)
```

TESTING DESIGN

- The project will focus on two types of testing: unit test and system tests
- Unit tests will be done through an automated testing suite in Github Actions
- System tests is a manual process and will involve two passes: one from the implementer and one from the QA lead
- Unit tests are responsible for testing back-end methods, front-end functions, and front-end components
- System tests are responsible for testing the entire user story and/or feature implementation

- Test items:
- Test priority: Medium
- Dependencies:
 -  [\[STORY\] User Creation and View #32](#) is complete
 -  [\[TASK\] Admin Should Be Able to Create New Users #36](#) is complete
 -  [\[TASK\] Admin Should Be Able to View All Users #37](#) is complete
- Preconditions:
 - An admin user should already exist within the system
- Test steps:
 - The user should login as an admin account
 - There should be a link (or menu item) that allows the admin to view all accounts
 - After navigating to the list of users. There should be search and filter functionality
 - Search for a name that is in the system. The return results should only contain records with the specified search result
 - Filter list for a specified role. The return results should only contain records with the specified role
 - There should be a Add New User button that takes the admin to a new page
 - Once there, add a new user and specified the role
 - After successfully adding, the admin should be redirected back to the view all accounts page
- Postconditions:
 - None
- Expected output:
 - The admin should be able to view all users that are currently within the system
 - The admin should be able to filter and sort the list of employees within the system
 - The admin should be able to add a new user to the system
 - The UI should have error validation
- Actual output:
- Pass or Fail:
- Bug id/link:  [\[STORY\] User Creation and View #32](#)
- Additional notes:

ITERATION 2 MILESTONES

- Iteration 1 was used to finalize the architecture, testing, configuration, and local development
- Iteration 2 will focus heavily on user story/feature work for all team members
- The goal is to finish 90% of the required user stories with the unit tests appropriately written and conducted
- All system tests should also be written for each user story and should began documenting at this time
- CI/CD pipeline should also be set-up and fully functional for development

